Appello 23 luglio 2015



Politecnico di Milano

Anno accademico 2014-2015

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri la seguente interfaccia Java `Albero<T>` che specifica i metodi pubblici di un ADT Albero. I nodi dell'albero sono oggetti di una classe immutabile, detta `Nodo<T>`, precisata nel seguito. Ogni nodo memorizza un valore di tipo `T`.

```
public interface Albero<T> {
//OVERVIEW: Un albero con radice, dalla quale esiste un unico cammino ad ogni altro nodo.
//Ogni nodo puo' avere 0, 1 o piu' figli, disposti in un ordine qualunque.
//L'albero non puo' essere vuoto. Ogni nodo e' etichettato con un valore di tipo T.
   //@ensures (* \result e' il numero di nodi dell'albero *);
   public /*@ pure @*/ int size();

   //@ensures (* \result e' la radice dell'albero *)
   public /*@ pure @*/ Nodo<T> radice();

   //@ensures (* \result e' un iteratore a tutti e soli i figli del nodo n
   //@  in ordine qualunque.*);
   public Iterator<Nodo<T> figli(Nodo<T> n);

   //@ensures (*restituisce il padre del nodo n, lanciando eccezione
   //@ se n e' la radice o se n non esiste nell'albero*);
   public /*@ pure @*/ Nodo<T> padre(Nodo<T> n) throws InvalidNodeException;

   //@ensures (* \result e' l'insieme di tutti e soli i nodi dell'albero *);
   public /*@ pure @*/ Set<Nodo<T>> nodi();

   //@ensures (* inserisce come figlio del nodo n un nuovo nodo  contenente x;
   //@  \result e' il nuovo nodo. Lancia eccezione se n non esiste nell'albero *);
   public Nodo<T> inserisci(Nodo<T> n, T x) throws InvalidNodeException;
}
```

Se utile, si consideri anche la seguente semplice implementazione di `Nodo<T>`:

```
public /*@ pure @*/ class Nodo<T> {
   private T valore;
   public Nodo(T x){valore = x;}
   public T getValore(return valore;)
}
```

Domanda a

Considerando come date le specifiche dei metodi puri `padre()`, `nodi()` e `radice()` descrivere in JML la specifica dei metodi `size()` e `inserisci()`. Definire anche, in base alla overview, un opportuno *invariante astratto*.

```
   //@ensures \result==nodi().size();
   public /*@ pure @*/ int size();
```

Il metodo inserisci deve garantire che il nodo creato (rappresentato da result) è effettivamente un nodo nuovo, di valore x, ed è figlio del nodo n (che deve essere parte dell'albero al momento della chiamata). Inoltre, il resto dell'albero non cambia, ossia ogni nodo esistente è ancora presente nell'albero ed è ancora figlio dello stesso padre. Il lancio dell'eccezione comporta che il nodo n non sia parte dell'albero al momento della chiamata; occorre anche segnalare che l'albero no cambia (con una condizione molto simile a quella della postcondizione normale).

```
   //@ensures n !=null && \old(nodi().contains(n)) &&
   //@        nodi().contains(\result) &&
   //@        !\old(nodi().contains(\result)) &&
   //@        \result !=null && \result.getValore().equals(x)) &&
```

```
   //@        padre(\result) == n &&
   //@        nodi().size()==1+\old(nodi().size()) &&
   //@        nodi().containsAll(\old(nodi()) &&
   //@        (\forall  Nodo<T> n1; \old(nodi().contains(n1));
   //@                  n1!= radice() ==> n1.padre() == \old(n1.padre());
   //@signals (InvalidNodeException e) (!\old(nodi().contains(n)) || n == null)  &&
   //@        nodi().equals(\old(nodi()) &&
   //@        (\forall  Nodo<T> n1; \old(nodi().contains(n1));
   //@                  n1!= radice() ==> n1.padre() == \old(n1.padre());
   public Nodo<T> inserisci(Nodo<T> n,T x);
```

Per esprimere le proprieta' della Overview è sufficiente scrivere nell'invariante che l'albero abbia almeno un nodo e che ogni nodo diverso dalla radice abbia un padre.

```
//@public invariant size()>0 &&
//@ (\forall  Nodo<T> n; nodi().contains(n);
//@           n == radice() || nodi().contains(padre(n));
```

Domanda b

Si consideri ora la specifica della seguente interfaccia:

```
public interface AlberoBinario<T>  extends Albero<T>  {
\\OVERVIEW: Un Albero in cui ogni nodo ha 0, 1 o 2 figli.
  //@ensures (*\result e' il figlio sx di n, se esiste*);
   public /*@ pure @*/ Nodo<T> sx(Nodo<T> n) throws InvalidNodeException;

  //@ensures (*\result e' il figlio dx di n, se esiste*);
   public /*@ pure @*/ Nodo<T> dx(Nodo<T> n) throws InvalidNodeException;
}
```

Un `AlberoBinario<T>` è sostituibile a un `Albero<T>` in base al principio di sostituzione di Liskov? La specifica dell'interfaccia è in grado di garantire il proprio invariante, così come descritto nella overview? Motivare la risposta.

SOLUZIONE: Si, in quanto regole della segnatura e dei metodi sono ovviamente verificate e la regola delle proprietà vale: un AlberoBinario è solo un caso particolare di un Albero. Tuttavia, la specifica della `inserisci()` stabilisce che è possibile inserire un numero arbitrario di figli per ogni nodo e quindi la proprietà invariante di essere un albero binario non può essere garantita.

Domanda c

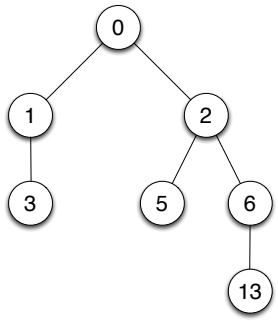
Sia data ora la seguente classe che implementa `AlberoBinario<T>`.

```
class AlbBinVettore<T> implements AlberoBinario<T> {
   private ArrayList<Nodo<T>> vettore;
   ...implementazione di tutti i metodi.
}
```

I nodi dell'albero binario sono memorizzati in un `Arraylist<Nodo><T> vettore` che ha le seguenti caratteristiche. È dato un metodo `p()` che per ogni nodo dell'albero ne calcola l'indice, numerando i nodi per livelli, da sinistra a destra, partendo da 0 per la radice: 1 e 2 per i suoi figli, poi 3 e 4 per i figli del nodo 1, ecc. Si noti che alcuni numeri possono essere saltati. Nelle posizioni che non corrispondono a un nodo, `vettore` vale `null`. Formalmente, `p$` è definita come segue:

- Se n è la radice dell'albero binario, $p(n) = 0$;
- Se n è il figlio sinistro di un nodo m, allora $p(n) = 2p(m) + 1$;
- Se n è il figlio destro di un nodo m, allora $p(n) = 2p(m) + 2$.

Quindi ad esempio:



Si scrivano invariante di rappresentazione e funzione di astrazione della classe `AlbBinVettore<T>`.
SOLUZIONE:

RI afferma che il vettore non ha nodi duplicati (altrimenti non sarebbe un albero) e che ogni nodo, salvo la radice, ha un padre. Il padre di un nodo di indice i ha indice $(i-1)/2$.

```
private invariant vettore!= null &&
        (\forall int i; 0\le i && i<vettore.size();
         vettore.get(i)!=null ==>
                i=0 || vettore.get((i-1)/2)!=null &&
                (\forall int j; i< j && j<vettore.size();
                         vettore.get(i)\neq vettore.get(j)));
```

AF: La funzione di astrazione produce la radice del nodo e stabilisce che `nodi()` restituisce tutti e soli i nodi di `vettore` che sono diversi da `null`.

```
private invariant radice()==vettore.get(0) &&
        vettore.containsAll(nodi()) &&
        (\forall Nodo<T> n; vettore.contains(n);
                n!=null ==> nodi().contains(n)) &&
        (\forall Nodo<T> n; nodi().contains(n);
                vettore.contains(n));
```

La versione con toString() puo' in questo caso essere piu' precisa e restituire un stringa in cui i nodi sono disposti nell'ordine per livelli:

```
@Override
public String toString() {
  String s = vettore.get(0).toString() &&
        for (int i=1; i<vettori.size();i++) {
```

```
     int k = l
             if (vettori.get(i)!=null) s= s+ " " + vettore.get(i).tostring()
        }
return s;
```

Si potrebbe anche inserire un carattere separatore fra ogni livello (quando i ha la forma 2^k-1 per un intero $k \geq 0$).

Esercizio 2

Si consideri la seguente classe Java

```
public class Counter {
   private int count = 0;
   void inc() { count++; }
   void dec() { count--; }
   int getCount() {return count;   }
}
```

Si consideri anche la seguente classe che genera threads che usano Counter

```
public class TroubleMaker implements Runnable {
   private Counter count;

   public void run() {
      for(int i=0; i<10000; i++){
         if ( i%2 == 0 )
             count.inc();
         else
             count.dec();
         System.out.println("Count is: " + count.getCount());
      }
   }

   public TroubleMaker(Counter c) {
      count = c;
   }

   public static void main(String args[]) {
      Counter c = new Counter();
      for(int i=1; i<11; i++)
         (new Thread(new TroubleMaker(c))).start();
   }
}
```

Riportare le modifiche che occorre apportare per far sì che

1. L'accesso al contatore avvenga in mutua esclusione tra i diversi thread
2. Eventuali thread che invocano il decremento del contatore vengano sospesi fino a che il contatore non diventa positivo, onde evitare che esso possa diventare negativo.
3. Si consideri l'operazione `println` effettuata da un thread. Così come è scritto il programma non è assicurato che il valore stampato sia il valore prodotto dall'incremento (o decremento) che il task ha effettuato con la predente istruzione. Si fornisca una sintetica spiegazione del perché. Si mostri anche quale modifica deve essere fatta al metodo run per assicurare che ciò invece avvenga.

SOLUZIONE:

1. Basta rendere synchronized tutti i metodi di Counter.

2. Si modifichi la classe Counter come segue:

```
public class Counter {

   private int count = 0;
   synchronized void inc() {
      count++;
      this.notifyAll();
   }
   synchronized void dec() {
      while(count==0) {
         try {
            this.wait();
         } catch (InterruptedException e) {
            e.printStackTrace();
         }
      }
      count--;
   }
   synchronized int getCount() {
      return count;
   }
}
```

3. A causa del nondetermismo, *count* potrebbe essere modificato da un altro thread subito dopo l'incremento o il decremento della variabile. Per ovviare al problema basta ad esempio che il metodo prenda un lock sull'oggetto `count`:

```
public void run() {
   for(int i=0; i<10000; i++){
      synchronized (count) {
         if ( i%2 == 0 )
            count.inc();
         else
            count.dec();
         System.out.println("Count is: " + count.getCount());
      }
   }
}
```

   Non serve invece a nulla rendere il metodo synchronized (come sempre per i metodi run). Si potrebbe invece definire un oggetto lock su cui run si sincronizza, ma solo se non vi sono altri thread che accedono a count.

Esercizio 3

Uno `Zoo` è composto da più gabbie. Una `Gabbia` può contenere animali. Esistono tre tipi di animali: `Tigre`, `Leone` e `Pappagallo`.

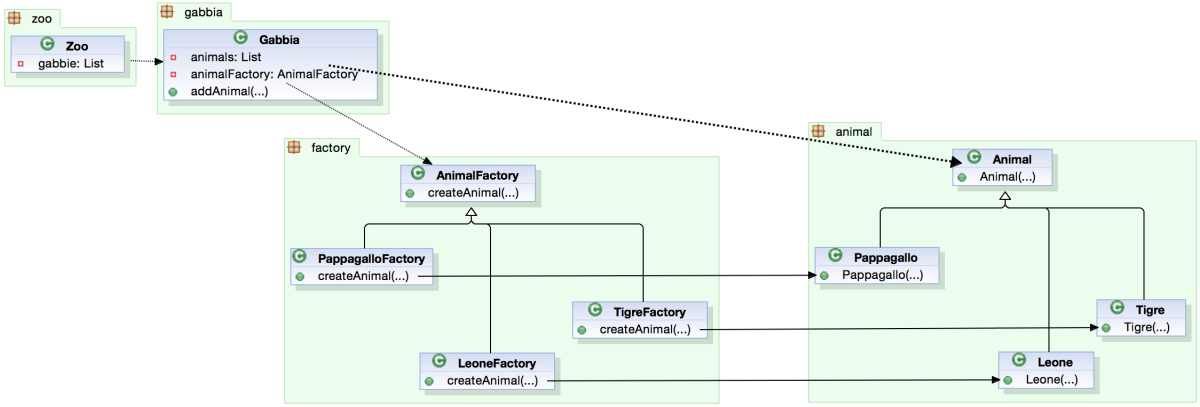
1. Fornire il diagramma delle classi UML che specifica il problema.

2. Si vuole garantire che una `Gabbia` possa contenere animali di uno stesso tipo, ma che questo tipo non sia noto a priori. Per questo si usa il pattern *factory method*. Quando viene invocato il costruttore di

`Gabbia`, viene passato come parametro la factory che crea animali del tipo voluto. Questa factory viene chiamata quando viene invocato il metodo `addAnimal` su una `Gabbia`.

Mostrare le aggiunte al diagramma UML che rappresentano quanto detto sopra.

3. Fornire l'implementazione completa della classe `Gabbia`: rappresentazione, costruttore e metodo `addAnimal`.




```
public class Gabbia {

   private List<Animal> animals = new ArrayList<Animal>();
   private AnimalFactory animalFactory;

   public Gabbia(AnimalFactory af) {
      animalFactory=af;
   }
   public void addAnimal(){
      animals.add(animalFactory.createAnimal());
   }

}
```

Esercizio 4

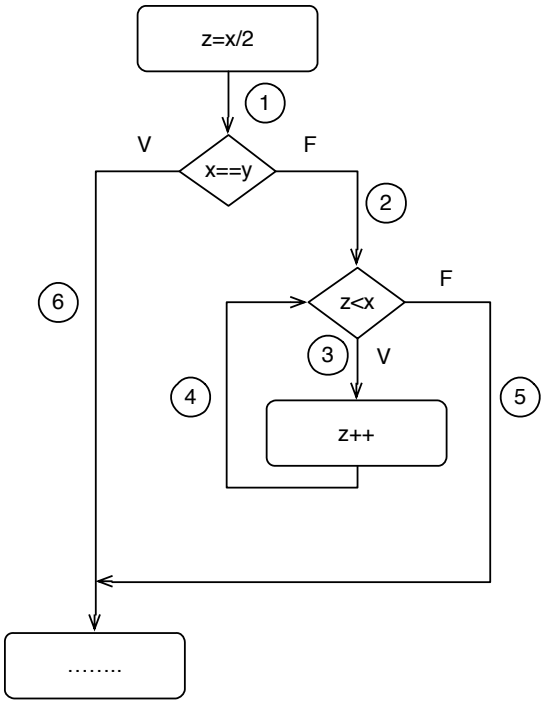
Si consideri questo frammento di programma che opera su interi:

```
static void method(int x, int y) {
   int z = x/2;
   if (x == y)
      ...
   else
      while (z < x)
         z++;
}
```

1. Si definisca un insieme di casi di test che copre tutti i *branch*
2. Si calcoli il predicato che deve essere soddisfatto da un caso di test che
   - Entra nel ramo else e percorre il ciclo 0 volte;
   - Entra nel ramo else e percorre il ciclo 1 volta;
   - Entra nel ramo else e percorre il ciclo 2 volte;
   - Entra nel ramo then.
3. Per ciascun predicato, si sintetizzi il caso di test che provoca la copertura del relativo cammino.

SOLUZIONE

Punto 1



Dopo aver rappresentato il control flow è necessario coprire i branch/edge marcati con ①, ②, ③, ④, ⑤, ⑥.
$x = 2, y = 2$ copre i branches ① and ⑥.
$x = 2, y = 1$ copre i branches ①, ②, ③, ④ and ⑤.

Punto 2a

$x \neq y \wedge x \leq 0$

Punto 2b
$x \neq y \wedge 1 \leq x \leq 2$
Punto 2c
$x \neq y \wedge 3 \leq x \leq 4$
Punto 2c
$x = y$
Punto 2d
$x = 0, y = 1$
$x = 2, y = 3$
$x = 4, y = 3$
$x = y = 1.$